

SGM Graph Filter Progress Update

Paper Reproduction, Torch Functions, and MLP Experiment

Harshil Shah

Mission San Jose High School

October 5, 2026

network_transfer_local_alpha

Propagation delay

$$\tau_{ij} = 10^{-3} D_{ij} / v$$

$$C_{ij}(\omega) = C_{ij} e^{-i\omega\tau_{ij}}$$

C : connection strengths

D : distances in mm

v : speed in m/s

```
C = brain.reducedConnectome
```

```
D = brain.distance_matrix
```

```
Tau = 0.001 * D / speed
```

```
Cc = C * np.exp(-1j * Tau * w)
```

`run_local_coupling_forward` calls this function at each $\omega = 2\pi f$.

network_transfer_local_alpha

Graph modes

$$L = I - \alpha \text{diag}(d)C(\omega)$$

$$Lu_k = \lambda_k u_k$$

d : inverse degree scaling

α : coupling

(λ_k, u_k) : eigenvalue and eigenvector

```
L1 = np.identity(nroi)
```

```
L = L1 - alpha * np.matmul(  
    np.diag(L2), Cc)
```

```
d, v = np.linalg.eig(L)
```

```
eig_ind = np.argsort(np.abs(d))
```

```
eig_vec = v[:, eig_ind]
```

```
eig_val = d[eig_ind]
```

```
eigenvectors = eig_vec[:, 0:K]
```

network_transfer_local_alpha

Local dynamics

$$F_e = \frac{1}{(1 + i\omega\tau_e)^2}, \quad F_i = \frac{g_{ii}}{(1 + i\omega\tau_i)^2}$$

$$H_{ed} = \frac{\alpha/\tau_e}{i\omega + (\alpha/\tau_e)F_e}$$

$$H_{id} = \frac{\alpha/\tau_i}{i\omega + (\alpha/\tau_i)F_i}$$

$$H_{eid} = \frac{g_{ei}F_eF_i}{1 + g_{ei}F_eF_i}$$

$$H_{\text{total}} = aH_{ed} + \frac{1-a}{2}(H_{id} + H_{eid})$$

$$Fe = \text{np.divide}(1 / \tau_e ** 2, \\ (1j * w + 1 / \tau_e) ** 2)$$

$$Fi = \text{np.divide}(g_{ii} * 1 / \tau_i ** 2, \\ (1j * w + 1 / \tau_i) ** 2)$$

$$Hed = \alpha / \tau_e / (\\ 1j * w + \alpha / \tau_e * Fe)$$

$$Hid = \alpha / \tau_i / (\\ 1j * w + \alpha / \tau_i * Fi)$$

$$Heid = g_{ei} * Fe * Fi / (1 + g_{ei} * Fe * Fi)$$

$$Htotal = (a * Hed + (1 - a) / 2 * Hid \\ + (1 - a) / 2 * Heid)$$

network_transfer_local_alpha

Regional response

$$q_k = \frac{i\omega\tau_C}{\alpha} + F_e\lambda_k$$

$$\tilde{q}_k = \max(|q_k|, 0.05 \max_j |q_j|) e^{i \arg(q_k)}$$

$$R_k = H_{\text{total}} / \tilde{q}_k$$

$$\hat{x}_r = \sum_{k=1}^{K-1} u_{rk} R_k$$

Mode 0 is omitted.

```
q1 = 1 / alpha * tauC * (  
    1j * w + alpha / tauC * Fe * eigenvalues)  
qthr = zero_thr * np.abs(q1[:]).max()  
magq1 = np.maximum(np.abs(q1), qthr)  
angq1 = np.angle(q1)  
q1 = np.multiply(magq1, np.exp(1j * angq1))  
frequency_response = np.divide(Htotal, q1)  
model_out = 0  
for k in range(1, K):  
    model_out += frequency_response[k] * (  
        eigenvectors[:, k])
```

Paper Reproduction

What I ran for this update

The forward calculation from `spectral_correlation.ipynb`.

Inputs

Saved parameter fits

Connectomes and MEG spectra

68 MEG regions, 40 frequencies



Outputs

Predicted regional spectra

Spectral correlations

36/36 matched the saved scores

Paper Reproduction

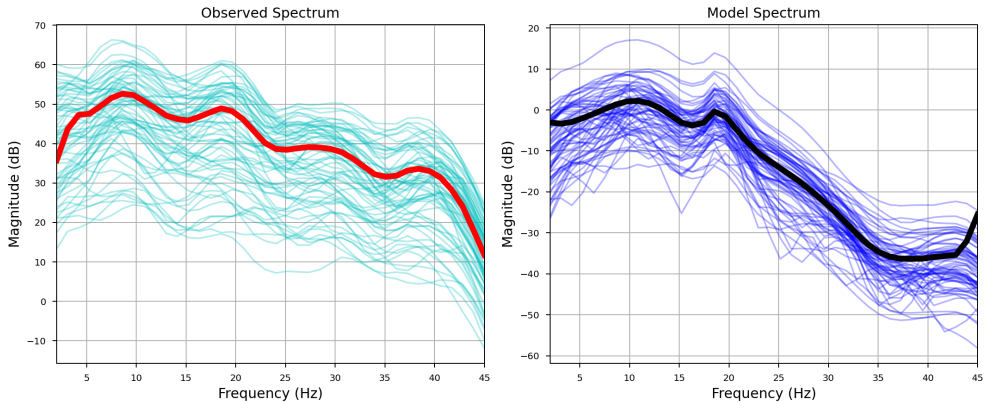


Figure 3 comparison: observed and modeled spectra from the replay.

Retained record 1, using the notebook plotting method.

What still needs to be confirmed?

Do we need a subject map to match these files?

Connectomes

39 records

No subject IDs

Fitted parameters

36 nonempty records

MEG spectra

36 nonempty records

Need to confirm which connectome goes with each MEG record and parameter set.

What still needs to be confirmed?

Region matching

82 structural nodes

(individual_subjects.mat)

86 model nodes

(OrderingAlphabetical_86ROIs.txt)

68 MEG regions

(freqMEGdata.mat)

Need the ordered region map.

Remaining reproduction

Fresh parameter fitting

Spatial alpha/beta results

Torch Functions

NumPy source

`network_transfer_local_alpha`

`run_local_coupling_forward`

Torch counterparts

`network_transfer_local_alpha_torch`

`run_local_coupling_forward_torch`

Inside the Torch forward function:

`prepare_graph_torch` computes the graph modes.

`evaluate_response_torch` computes the response.

Torch Functions

prepare_graph_torch

$$C_{ij}(\omega) = C_{ij} e^{-i\omega\tau_{ij}}$$

$$L = I - \alpha \text{diag}(d) C(\omega)$$

$$L u_k = \lambda_k u_k$$

```
Tau = 0.001 * D / speed
```

```
Cc = C * torch.exp(-1j * Tau * w)
```

```
values, vectors = torch.linalg.eig(L)
```

```
order = torch.argsort(torch.abs(values))
```

```
eigenvalues = values[order]
```

```
eigenvectors = vectors[:, order][:, :K]
```

Same degree scaling and mode order. Each eigenvector's largest entry is made positive real to fix its phase.

Torch Functions

evaluate_response_torch

$$F_e^{\text{local}} = \frac{1}{(1 + i\omega\tau_e)^2}$$

$$H_{ed} = \frac{\alpha/\tau_e}{i\omega + (\alpha/\tau_e)F_e^{\text{local}}}$$

$$H_{eid} = \frac{g_{ei}F_e^{\text{local}}F_i}{1 + g_{ei}F_e^{\text{local}}F_i}$$

$$\text{Fe_local} = (1 / \text{tau_e}^{**2}) / (\\ 1j * w + 1 / \text{tau_e}) ** 2$$

$$\text{Hed} = (\text{alpha} / \text{tau_e}) / (\\ 1j * w + \text{alpha} / \text{tau_e} * \text{Fe_local})$$

$$\text{Heid} = g_{ei} * \text{Fe_local} * F_i / (\\ 1 + g_{ei} * \text{Fe_local} * F_i)$$

The local excitatory and inhibitory equations keep the NumPy algebra.

Torch Functions

Local dynamics

$$F_e^{\text{local}}(\omega) = \frac{1}{(1 + i\omega\tau_e)^2}$$

Enters H_{ed} and H_{eid} .

Uses the subject's fitted τ_e .

Graph propagation

$$q_k = \frac{i\omega\tau_C}{\alpha} + F_e^{\text{graph}}\lambda_k$$

```
if Fe_graph is None:
```

```
    Fe_graph = Fe_local
```

```
q1 = (tauC / alpha) * (  
    1j * w + alpha / tauC * Fe_graph  
    * graph["eigenvalues"])
```

The MLP will replace Fe_graph . Local dynamics stay analytical.

MLP Implemented So Far

`training/filters.py`

`generate_training_data`

Saves graph modes, parameters and MEG.

`FeGraphMLP`

Maps frequency to one complex filter value.

`predict_spectra`

Uses the saved graphs and filter values.

`training/train.py`

`train_filter`

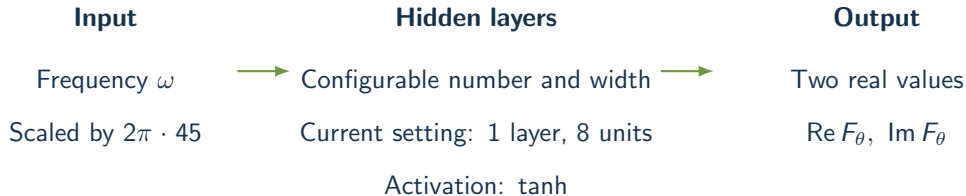
Trains the shared MLP.

Validation selects the saved weights.

Saves loss history and split scores.

MLP Implemented So Far

FeGraphMLP

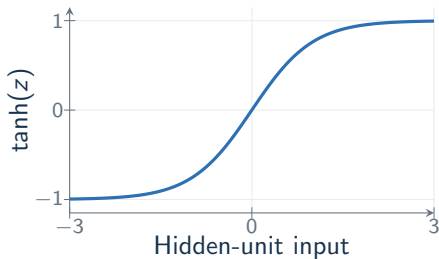


$$F_\theta(\omega) = \text{Re } F_\theta(\omega) + i \text{Im } F_\theta(\omega)$$

One learned complex graph-filter value per frequency, shared across subjects.

Preliminary Implementation Choices

tanh activation



Smooth hidden activation. The final linear layer leaves the output unbounded.

Starter architecture

1 input: scaled frequency

1 hidden layer: 8 units

2 outputs: real and imaginary parts

34 trainable parameters

Frequency scale: $2\pi \cdot 45$ rad/s

Adam, learning rate 10^{-3}

100 epochs, seed 7

MLP Training

train_filter

$$\mathcal{L} = 1 - \frac{1}{S} \sum_{s=1}^S r_s$$

r_s : mean regional spectral correlation

S : number of training subjects

Only the MLP weights are updated.

Graph modes are prepared once.

```
optimizer.zero_grad()
train_scores = correlations(
    splits["train"], model(omega))
loss = 1 - train_scores.mean()
loss.backward()
optimizer.step()
```

Planned Experiment

Analytical graph filter

$$F_{e,s}(\omega) = \frac{1}{(1 + i\omega\tau_{e,s})^2}$$

Each subject's fitted time constant.

Learned graph filter

$$F_{\theta}(\omega) \in \mathbb{C}$$

One MLP shared across subjects.

Keep the subjects, SGM parameters, local dynamics and scoring fixed.

Compare spectral correlation on held-out subjects.